MODULE 10 · DOMAIN 1 - DATA PREPARATION FOR ML AND AI

Preparing documents for RAG

Parsing, chunking and metadata. The work that decides whether retrieval works.

MODULE 10 of 43 DOMAIN 1 - Data Prep WEIGHT 28%

admin@sthithapragna.com

You retrieve **chunks**, not documents. So a chunk is the unit of usefulness.

From a file to a searchable chunk

01
Parse
Get text out of the file, including tables and images.

02
Clean
Strip boilerplate, fix encoding, normalise whitespace.

03
Chunk
Split into retrievable units, with overlap.

04
Enrich
Attach metadata: source, date, permissions, section.

05
Embed
Vector per chunk, into the store from module 09.

ON THE EXAM

Bedrock Knowledge Bases performs all five steps for you. You configure them rather than build them.

Small chunks are **precise and contextless.**
Large chunks are **rich and imprecise.**

Chunking strategies in Bedrock Knowledge Bases

STRATEGY	WHAT IT DOES
Default	Fixed-size chunking at 300 tokens with 20 percent overlap
Fixed size	The same, with the token count and overlap set by you
Semantic	Splits where meaning shifts, so chunks follow topic boundaries
Hierarchical	Small child chunks for searching, larger parent chunks returned
None	One chunk per file. For pre-split content, or a custom Lambda

ON THE EXAM

Learn the default numbers. 300 tokens with 20 percent overlap is what you get if you configure nothing.

Fixed-size chunking, and why it is usually enough

- Split every N tokens, regardless of what the text is saying. Simple and predictable.
- It will cut sentences and tables in half. That is the known cost.
- **Overlap repairs most of that damage**, which is why the default includes 20 percent.
- Cheap to run, easy to reason about, and a fine baseline for most corpora.
- Benchmarks routinely show it within a few percent of far more elaborate strategies.

> ON THE EXAM
>
> Start here. Move to something cleverer only when you have measured that fixed size is failing you.

Overlap, and how much of it

- Each chunk repeats the last part of the previous one, so ideas spanning a boundary survive.
- **10 to 20 percent is the usual range.** The Bedrock default is 20 percent.
- More overlap means more chunks, so more storage, more embedding cost, more query cost.
- It also means near-duplicate chunks competing in the same result list.
- Too little overlap and answers get truncated at boundaries you cannot see.

> ON THE EXAM
>
> Overlap is insurance against a boundary landing badly. Like all insurance, you can buy too much of it.

Chunk size and the prompt token budget

- Retrieved chunks are **pasted into the prompt**. Their size times top-k is your context cost.
- Ten chunks of 2,000 tokens is 20,000 tokens on **every single request**.
- Token cost is per request and permanent. Chunk size is therefore an ongoing cost decision.
- Larger chunks usually mean you can retrieve **fewer of them**, which partly offsets it.
- Overrunning the context window truncates silently, and the truncated part is often the answer.

> ON THE EXAM
>
> Chunk size sets storage cost once and inference cost forever. The second is usually the larger number.

Semantic chunking

- Embeds sentences, then splits where **consecutive sentences stop being similar**.
- Chunks therefore follow topic boundaries rather than arbitrary token counts.
- Defaults: buffer size 0, breakpoint percentile threshold 95, max tokens 300.
- **It embeds during ingestion to decide the splits**, so ingestion is slower and costlier.
- On a very large corpus that ingestion cost becomes the deciding factor.

> ON THE EXAM
>
> It buys better boundaries with a more expensive ingestion. Worth it on prose with clear topic shifts, less so on uniform text.

Hierarchical chunking

- Creates **small child chunks** and **larger parent chunks** containing them.
- **Search runs on the children. The parent is what gets returned.**
- So you get the precision of small chunks with the context of large ones.
- **Expect fewer results**, because one parent covers several matching children.
- Best for nested documents: technical manuals, legal documents, academic papers.

> ON THE EXAM
>
> This is the strategy that resolves the central trade-off rather than compromising on it. It is also the one people forget exists.

No chunking, and custom Lambda chunking

- **No chunking** treats each file as one chunk. Correct when files are already small and single-topic.
- It is also how you plug in **custom chunking logic via a Lambda function**.
- You supply an S3 bucket the knowledge base uses to write and read chunks.
- Worth it when a format has meaningful units the generic chunkers destroy.
- You now own a Lambda, its IAM, its failures and its cost. Do not reach for it casually.

> ON THE EXAM
>
> Source code, subtitle files and structured records all have natural boundaries a token counter cannot see.

Which chunking strategy

WHAT THE SCENARIO SAYS	THE STRATEGY
Nothing special. General prose, get started	Default, or fixed size
Retrieved passages lack surrounding context	Hierarchical
Nested structure: manuals, contracts, papers	Hierarchical
Topics shift within documents, boundaries matter	Semantic
Files are already short and single-topic	None
A domain format with units a token count destroys	None, plus a custom Lambda

ON THE EXAM

'The answers are fragments without context' is the hierarchical signal. It appears almost word for word.

Testing a chunking strategy

- Build a **small evaluation set**: real questions, with the passages that should answer them.
- Ingest a representative sample under two strategies, into two data sources.
- Compare **recall at k** from module 09. Did the right passage appear in the top results?
- Judging by reading a few answers feels convincing and measures almost nothing.
- Published benchmarks often find fixed size within a few percent of elaborate strategies.

> ON THE EXAM
>
> Because chunking cannot be changed later, the sample test is the only cheap moment to get this right.

Chunking is fixed once the data source exists

- **You cannot change the chunking strategy after connecting a data source.**
- Changing it means creating a **new data source** and re-ingesting the content.
- Same shape as module 09: index dimension and metric are also fixed at creation.
- So test chunking on a sample before ingesting a large corpus.
- Everything downstream is derived state, so re-ingestion is a job rather than a disaster.

> ON THE EXAM
>
> Two irreversible decisions now sit at the start of a RAG pipeline: the embedding model, and the chunking strategy.

A PDF is not text. Something has to get the words out, in the right order.

What each one can handle

PARSER	HANDLES
Default Bedrock parser	Text only. No additional cost
Foundation model parser	PDFs, images, tables, visually rich documents. Not audio or video
Bedrock Data Automation	PDFs, images, and audio and video as well

ON THE EXAM

Audio and video are the dividing line. Only BDA handles them, and the FM parser explicitly does not.

Bedrock Data Automation

- Generally available since **March 2025**. Usable standalone, or as a Knowledge Bases parser.
- **Standard output:** text in reading order, summaries, captions for tables, figures and diagrams.
- **Custom output uses blueprints:** fields to extract plus a format, one blueprint per document type.
- Transcribes audio and video, and describes visual content.
- Limits: PDF, PNG, JPG, TIFF. Max **1,500 pages** and **500 MB** per API request.

> ON THE EXAM
>
> The Knowledge Bases integration uses standard output. Blueprints are for extracting specific fields in your own pipeline.

CSV and tabular sources

- Knowledge Bases can treat CSV columns as **content or as metadata**, rather than embedding raw rows.
- That means a product table becomes chunks with real filterable attributes attached.
- Embedding a raw CSV row produces a vector of commas and codes with almost no meaning.
- Prefer generating a short natural-language description per row, then embedding that.
- Genuinely tabular questions belong in Athena or SQL, not in a retrieval system.

> ON THE EXAM
>
> Embeddings capture meaning, and a comma-separated row barely has any. Give the model a sentence to embed.

Smart Parsing, and the option it removes

- **Smart Parsing** picks the parsing strategy per data type automatically, with no configuration.
- It preserves metadata, embedded images and thread context from connectors.
- **Managed knowledge bases support SMART_PARSING only.**
- The foundation model parser and BDA are **not available** in a managed knowledge base.
- So choosing managed trades parser control for far less configuration.

> ON THE EXAM
>
> This is a real fork. If a scenario requires a specific parser, the managed option is already ruled out.

Nova Multimodal Embeddings or BDA?

NOVA MULTIMODAL, WHEN

Visual similarity matters

- Search images by image, or by description.
- Product catalogues and visual archives.
- Chunk duration 1-30 seconds, default 5.
- Limited support for speech content.

BDA AS PARSER, WHEN

What was said matters

- Meeting recordings, calls, training videos.
- Full speech transcription, searchable as text.
- Also describes the visual content.
- Converts everything to text representations.

THE TIEBREAKER

Combine them and BDA parsing wins: Nova then behaves as an ordinary text embedding model.

Tables, and why they break retrieval

- A table split across two chunks loses its header row, so the numbers become meaningless.
- Flattened to text in reading order, columns interleave and rows stop lining up.
- The FM parser and BDA both **generate captions** for tables, which embed far better than raw cells.
- Keeping a table whole in one chunk usually beats any clever splitting of it.
- For heavily tabular data, consider whether retrieval is even the right tool. SQL may be.

> ON THE EXAM
>
> A number without its column header is not information. That is the whole problem with chunked tables.

Scanned documents and OCR

- A scanned PDF contains no text at all. The default parser finds nothing to extract.
- **Amazon Textract** extracts text, forms and tables from scans, as covered in module 01.
- BDA and the FM parser also handle scans, and additionally describe figures and diagrams.
- OCR errors propagate into embeddings silently. A misread digit becomes a wrong retrieved fact.
- Handwriting remains the hardest case, and is where human review from module 07 earns its place.

> ON THE EXAM
>
> 'Content in the PDF is never retrieved' is usually a scanned page nobody realised was an image.

Document hygiene

- **Strip repeated boilerplate.** Headers, footers and disclaimers on every page add noise to every chunk.
- Normalise whitespace and fix character encoding, or chunks carry rubbish into the vector.
- Remove navigation and markup from crawled web content.
- **Do not stem or remove stop words.** Module 05: that harms embeddings rather than helping.
- Deduplicate near-identical documents, or the same answer fills your entire top k.

> ON THE EXAM
>
> Boilerplate is the quiet killer. A legal disclaimer repeated on 4,000 pages makes 4,000 chunks slightly more similar to each other.

Access control has to be built in now

- Permissions must be captured as **metadata at ingestion**, then filtered at query time.
- Metadata you did not capture cannot be filtered on later without re-ingesting.
- Filtering **after** retrieval is not equivalent: you get too few results, and the data still left the store.
- Connectors can carry the source system's permission model, which is often the reason to use one.
- Anything you would not show every user needs a filterable attribute on every chunk.

> ON THE EXAM
>
> Retrieval bypasses the application's own access control. If the chunk does not carry permissions, nothing is enforcing them.

Metadata extraction

- **Provenance:** source document, page or section, so answers can be cited.
- **Time:** published and last-modified dates, so stale content can be filtered or down-weighted.
- **Access:** owning team, classification, permissions. Filtering here is your security boundary.
- **Structure:** document type and section heading, which improve filtering and ranking.
- Metadata not captured at ingestion cannot be recovered later without re-ingesting.

> ON THE EXAM
>
> Citation, which module 01 called the giveaway for choosing retrieval, is only possible because the chunk remembers where it came from.

Query reformulation

- A complex question is **broken into sub-queries**, each retrieved separately.
- 'Compare our refund policy in the UK and Germany' becomes two retrievals, not one bad one.
- It also rewrites vague or conversational phrasing into something retrievable.
- It costs an extra model call before retrieval, so it adds latency and tokens.
- Announced alongside advanced parsing and chunking as a Knowledge Bases accuracy feature.

> ON THE EXAM
>
> A multi-part question embedded whole matches nothing well, because its vector sits between two topics.

Data sources and connectors

- Amazon S3 is the common source, and the one you control most directly.
- Managed knowledge bases connect to **SharePoint, Confluence, Google Drive, OneDrive** and a web crawler.
- Connectors preserve source-specific structure such as thread context, which plain files lose.
- Crawled web content needs the heaviest cleaning of any source.
- Ingestion jobs handle change detection, so unchanged documents are not re-embedded.

> ON THE EXAM
>
> Connectors also carry the source's permissions model. That is often the reason to use one rather than exporting to S3.

Ingestion jobs, updates and deletions

- An **ingestion job** syncs a data source. Change detection means unchanged files are skipped.
- Changed documents are re-chunked and re-embedded. Deleted documents have their vectors removed.
- **A document deleted at source but never synced stays retrievable.** That is a real disclosure risk.
- Sync frequency is a policy decision: how stale may an answer be before it is wrong?
- Module 28 covers scheduling these refresh cycles as part of a pipeline.

> ON THE EXAM
>
> A retracted document that is still retrievable is worse than one that was never indexed. Sync cadence is a governance decision.

What preparing documents costs

COMPONENT	COST SHAPE
Default parser	No additional charge. Text only
Foundation model parser	Per page or per document, using a model to interpret layout
Bedrock Data Automation	Per page, image, or minute of audio and video processed
Semantic chunking	Extra embedding calls during ingestion to decide the splits
Custom Lambda chunking	Lambda invocations, plus the S3 round trip for chunks
Embedding	Per input token, once per chunk. Overlap multiplies this
Re-ingestion	The whole ingestion cost again, whenever a strategy changes

ON THE EXAM

Ingestion is a one-off cost you pay again on every strategy change. That is the argument for testing on a sample.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
Answers are fragments missing context	Chunks too small. Try hierarchical
Retrieval returns vaguely related passages	Chunks too large, so each one is diluted
Numbers in answers are wrong or unattached	A table split across chunks, losing its headers
Everything retrieved looks slightly similar	Boilerplate repeated across every chunk
Content in a PDF is simply never retrieved	It was in an image or table the default parser skipped
Cannot change chunking on a data source	Fixed at creation. New data source, re-ingest
Fewer results than top-k requested	Hierarchical chunking collapsing children into parents

ON THE EXAM

Rows one and two are opposites, and both are chunk-size problems. The complaint tells you which direction to move.

A chunk must be able to answer a question **on its own**. That is the whole test.

> An engineering firm builds a knowledge base over 12,000 equipment maintenance manuals. They are deeply nested: numbered sections, sub-procedures and cross-references.
>
> Using the default chunking, retrieved passages are individual steps such as 'Torque to 45 Nm and reassemble', with no indication of which component or procedure they belong to. Answers are confidently wrong.

Which chunking strategy should they use?

A Fixed size, increased to 2,000 tokens with 20 percent overlap

B Semantic chunking, so splits follow topic boundaries

C Hierarchical chunking, searching child chunks and returning parents

D No chunking, treating each manual as a single chunk

◆ PAUSE AND DECIDE

Which chunking strategy should they use?

A. Fixed size, increased to 2,000 tokens with 20 percent overlap

Adds context and loses precision. Large chunks are diluted, so matching gets worse even as the missing context problem improves.

B. Semantic chunking, so splits follow topic boundaries

Better boundaries than fixed size, and each chunk is still returned alone. A well-bounded step still does not say which procedure it belongs to.

C. Hierarchical chunking, searching child chunks and returning parents

Matches the precise step, returns the enclosing procedure. AWS names technical manuals as its case, and nesting is exactly what it exploits.

D. No chunking, treating each manual as a single chunk

One vector per manual represents everything and matches nothing sharply. It also will not fit in a prompt.

THE SIGNAL

'Precise but lacking context' is the hierarchical signal. Only hierarchical improves both sides; every other option trades one for the other.

A consultancy stores 4,000 recorded client calls and 900 training videos in Amazon S3. They want staff to ask questions and get answers drawn from what was said in those recordings.

An engineer configures a Bedrock knowledge base with the foundation model parser, reasoning that it handles rich content. After ingestion, no recording content is retrievable.

What should be changed?

A Use Bedrock Data Automation as the parser, which transcribes audio and video

B Switch the embedding model to Amazon Nova Multimodal Embeddings V1.0

C Change the chunking strategy to semantic so speech boundaries are respected

D Enable Smart Parsing on the existing data source

◆ PAUSE AND DECIDE

What should be changed?

A Use Bedrock Data Automation as the parser, which transcribes audio and video

BDA is the only parser that handles audio and video. It transcribes speech into searchable text, which is precisely what was asked for.

B Switch the embedding model to Amazon Nova Multimodal Embeddings V1.0

Nova gives visual and audio similarity search, and AWS documents limited support for speech content. For speech, it points you at BDA.

C Change the chunking strategy to semantic so speech boundaries are respected

Chunking operates on text that a parser produced. There is no text here, so no chunking strategy can help.

D Enable Smart Parsing on the existing data source

Smart Parsing belongs to managed knowledge bases, which support it exclusively. It is not a toggle on a data source using the FM parser.

THE SIGNAL

The FM parser handles PDFs, images and tables, and explicitly not audio or video. That single boundary decides the question.

> A team ingested 60,000 support articles into a Bedrock knowledge base using the default chunking strategy six weeks ago. Evaluation now shows that semantic chunking would improve retrieval accuracy measurably on their corpus.
>
> They open the data source configuration to change the chunking strategy and find they cannot.

What is the correct course of action?

A Delete and re-upload the source documents, which resets the chunking configuration

B Create a new data source with semantic chunking and re-ingest the articles

C Raise a support request, since chunking strategy is a soft limit

D Add a second chunking configuration, so both strategies run in parallel

◆ PAUSE AND DECIDE

What is the correct course of action?

A. Delete and re-upload the source documents, which resets the chunking configuration

Chunking is a property of the data source, not the documents. Re-uploading files re-ingests them with the same strategy.

B. Create a new data source with semantic chunking and re-ingest the articles

Chunking cannot be changed after a data source is connected. A new data source is the documented path, and the source articles are still in S3.

C. Raise a support request, since chunking strategy is a soft limit

This is a design constraint, not a quota. Chunks were already written and embedded under the old strategy; support cannot rewrite them.

D. Add a second chunking configuration, so both strategies run in parallel

A data source has one chunking configuration. Two strategies means two data sources, which is what option B describes.

THE SIGNAL

The same shape as module 09's fixed index settings. Test on a sample first, because these choices are made once.

A retailer wants shoppers to upload a photograph of an item and find visually similar products in a catalogue of 200,000 product images with short text descriptions.

There is no audio or video, and no requirement to search spoken content. The team is choosing how to configure a multimodal knowledge base.

Which processing approach fits?

A Bedrock Data Automation as parser with Titan Text Embeddings V2

B The default Bedrock parser with Titan Text Embeddings V2 on the descriptions

C Bedrock Data Automation as parser with Amazon Nova Multimodal Embeddings

D Amazon Nova Multimodal Embeddings V1.0, without a BDA parser

◆ PAUSE AND DECIDE

Which processing approach fits?

A. Bedrock Data Automation as parser with Titan Text Embeddings V2

BDA converts images to text descriptions, then a text model embeds that text. You get search over descriptions, not visual similarity.

B. The default Bedrock parser with Titan Text Embeddings V2 on the descriptions

Text-only retrieval. A shopper's photograph cannot be turned into a query at all, which is the entire requirement.

C. Bedrock Data Automation as parser with Amazon Nova Multimodal Embeddings

The documented trap: with BDA parsing in place, Nova acts as a text embedding model, so the multimodal capability is lost.

D. Amazon Nova Multimodal Embeddings V1.0, without a BDA parser

Images embedded directly into a shared visual space, so a photograph becomes a query. AWS names product catalogues as this approach's case.

THE SIGNAL

BDA converts multimodal content to text. That is its strength for speech and precisely wrong when you need visual similarity.

What to take from this module

- **Default is 300 tokens with 20 percent overlap.** Know it, and start there.
- **Hierarchical** answers 'precise but no context'. Search children, return parents.
- **Chunking is fixed at data source creation.** Test on a sample before ingesting at scale.
- **Only BDA handles audio and video.** The FM parser does not.
- **A chunk must answer a question on its own.** That single test decides most of this.

> ON THE EXAM
>
> Module 11 closes Domain 1: preparing data for fine-tuning and distillation.